JAVA STACK, QUEUE & HEAP INTERVIEW HANDBOOK

Stack • Queue • Deque • PriorityQueue • Monotonic Stack/Queue • Heap Patterns

Concepts + formulas + Java code + dry runs + complexity + interview problem bank

Java | IntelliJ IDEA | Interview-focused

How to use this guide

• Create one Java class per problem in IntelliJ. File name and public class name must match.

• Run each code example independently. Examples use standard Java collections and include main() where a standalone demo is useful.

• Learn the invariant (what must remain true after every operation), not only the syntax.

• Complexities are average/amortized where stated; worst-case behavior can differ.

• This guide gives full runnable implementations for core patterns and a practice bank. The practice bank is not a claim that every listed problem has full code in this PDF.

Imports used repeatedly: `import java.util.*;` and, where needed, `import java.io.*;`

1. Big picture: which structure should I choose?

Structure	Rule / behavior	Best use
Stack	LIFO: last in, first out	Undo, parsing, DFS, next greater element
Queue	FIFO: first in, first out	BFS, scheduling, level order
Deque	Insert/remove at both ends	Sliding window, 0-1 BFS
PriorityQueue	Highest/lowest priority first	Top-K, Dijkstra, scheduling, merging
Monotonic stack	Stack kept increasing/decreasing	Next greater/smaller, histogram
Monotonic deque	Deque kept monotonic	Sliding window maximum/minimum
Heap	Complete binary tree with heap order	Priority queue implementation, streaming Kth

2. Stack — fundamentals

Definition: Stack is a linear data structure that follows LIFO (Last In, First Out). The most recently inserted item is removed first.

Core operations: push(x), pop(), peek(), isEmpty(), size(). In Java, prefer `Deque` with `ArrayDeque` for normal stack work.

• push(x): add to top — O(1) amortized

• pop(): remove top — O(1) amortized

• peek(): read top — O(1)

• isEmpty()/size(): O(1)

• Avoid legacy Stack class for new code; ArrayDeque is usually the better choice.

Runnable stack operations

```
import java.util.*;

public class StackBasics {
    public static void main(String[] args) {
        Deque<Integer> st = new ArrayDeque<>();

        st.push(10); // top = 10
        st.push(20); // top = 20
        st.push(30); // top = 30

        System.out.println(st.peek()); // 30
        System.out.println(st.pop());  // 30
        System.out.println(st.peek()); // 20
        System.out.println(st.isEmpty()); // false
        System.out.println(st.size()); // 2
    }
}
```

Important Java distinction: `push/pop/peek` operate at the front of the Deque. `addFirst/removeFirst/peekFirst` are equivalent front-end operations. Do not call pop on an empty stack; it throws NoSuchElementException.

3. Stack formulas, patterns and invariants

• Balanced brackets: push opening brackets; on a closing bracket, stack must be non-empty and top must be its matching opener; at end stack must be empty.

• Next Greater Element to right: scan right-to-left; pop while stack top <= current; answer is top or -1; then push current.

• Next Smaller Element to right: pop while top >= current.

• Previous Greater Element: scan left-to-right; pop while top <= current.

• Histogram rectangle: when a bar is popped, width = currentIndex - indexAfterNewStackTop - 1; area = height[popped] × width.

• Expression evaluation: operands push; operators pop operands in correct order (right operand first, then left).

• Time pattern: each element is pushed once and popped at most once ⇒ O(n) total, even if there is a while loop inside the for loop (amortized analysis).

Balanced parentheses — full code

```
import java.util.*;

public class ValidParentheses {
    static boolean isValid(String s) {
        Deque<Character> st = new ArrayDeque<>();

        for (char c : s.toCharArray()) {
            if (c == '(' || c == '[' || c == '{') {
                st.push(c);
            } else {
                if (st.isEmpty()) return false;
                char open = st.pop();

                if (c == ')' && open != '(') return false;
                if (c == ']' && open != '[') return false;
                if (c == '}' && open != '{') return false;
            }
        }
        return st.isEmpty();
    }

    public static void main(String[] args) {
        System.out.println(isValid("{[()]}")); // true
        System.out.println(isValid("{[(])}")); // false
    }
}
```

Dry run: For `{[()]}`: push {, [, (; ')' matches '(' and pops; ']' matches '['; '}' matches '{'; stack ends empty ⇒ valid.

Min Stack — O(1) getMin

Maintain a second stack of minimum values. Push x to minStack when it is <= current minimum. When popping the main stack, also pop minStack if the removed value equals its top.

```
import java.util.*;

public class MinStack {
    private final Deque<Integer> values = new ArrayDeque<>();
    private final Deque<Integer> mins = new ArrayDeque<>();

    public void push(int x) {
        values.push(x);
        if (mins.isEmpty() || x <= mins.peek()) mins.push(x);
    }

    public int pop() {
        if (values.isEmpty()) throw new NoSuchElementException();
        int x = values.pop();
        if (x == mins.peek()) mins.pop();
        return x;
    }

    public int top() {
        if (values.isEmpty()) throw new NoSuchElementException();
        return values.peek();
```

```
    }

    public int getMin() {
        if (mins.isEmpty()) throw new NoSuchElementException();
        return mins.peek();
    }

    public static void main(String[] args) {
        MinStack s = new MinStack();
        s.push(5); s.push(2); s.push(2); s.push(7);
        System.out.println(s.getMin()); // 2
        s.pop(); s.pop();
        System.out.println(s.getMin()); // 5
    }
}
```

Complexity: push/pop/top/getMin = O(1); extra space O(n).

4. Queue — fundamentals

Definition: Queue follows FIFO (First In, First Out). The first inserted element is removed first.

Operations: offer/add (insert at rear), poll/remove (remove front), peek/element (read front). Prefer offer/poll/peek because they return a special value instead of throwing on failure/empty.

• offer(x): O(1) amortized for ArrayDeque / LinkedList

• poll(): O(1)

• peek(): O(1)

• isEmpty()/size(): O(1)

• ArrayDeque does not allow null elements.

Queue using ArrayDeque

```
import java.util.*;

public class QueueBasics {
    public static void main(String[] args) {
        Queue<Integer> q = new ArrayDeque<>();

        q.offer(10);
        q.offer(20);
        q.offer(30);

        System.out.println(q.peek()); // 10
        System.out.println(q.poll()); // 10
        System.out.println(q.peek()); // 20
        System.out.println(q.size()); // 2
    }
}
```

Queue using linked list nodes — implementation

```
public class MyQueue {
    private static class Node {
        int val;
        Node next;
        Node(int val) { this.val = val; }
    }

    private Node front, rear;
    private int size;

    public void offer(int x) {
        Node node = new Node(x);
        if (rear == null) front = rear = node;
        else { rear.next = node; rear = node; }
        size++;
    }

    public int poll() {
        if (front == null) throw new java.util.NoSuchElementException();
        int x = front.val;
        front = front.next;
```

```
        if (front == null) rear = null;
        size--;
        return x;
    }

    public int peek() {
        if (front == null) throw new java.util.NoSuchElementException();
        return front.val;
    }

    public int size() { return size; }
    public boolean isEmpty() { return size == 0; }

    public static void main(String[] args) {
        MyQueue q = new MyQueue();
        q.offer(4); q.offer(8);
        System.out.println(q.poll()); // 4
        System.out.println(q.peek()); // 8
    }
}
```

Invariant: front points to the next element to remove; rear points to the last element. When the last node is removed, set rear = null too.

5. Deque (double-ended queue)

Deque allows insertion and deletion at both front and rear. Java: `Deque<Integer> dq = new ArrayDeque<>();`

• addFirst(x), offerFirst(x): insert at front

• addLast(x), offerLast(x): insert at rear

• removeFirst(), pollFirst(): remove front

• removeLast(), pollLast(): remove rear

• peekFirst(), peekLast(): inspect ends

Deque demo

```
import java.util.*;

public class DequeBasics {
    public static void main(String[] args) {
        Deque<Integer> dq = new ArrayDeque<>();
        dq.offerFirst(20); // [20]
        dq.offerLast(30);  // [20, 30]
        dq.offerFirst(10); // [10, 20, 30]

        System.out.println(dq);            // [10, 20, 30]
        System.out.println(dq.pollLast()); // 30
        System.out.println(dq.pollFirst());// 10
    }
}
```

Use cases: sliding window maximum/minimum, palindrome checks, 0-1 BFS, implementing stack/queue.

6. PriorityQueue and Heap

PriorityQueue: removes the element with the highest priority first. Java's default PriorityQueue is a min-heap, so the smallest element comes out first.

• offer/add: O(log n)

• poll/remove head: O(log n)

• peek: O(1)

• remove(Object): O(n)

• contains(Object): O(n)

• Building a heap from a collection is O(n) with heapify (implementation detail).

Min-heap and max-heap

```
import java.util.*;

public class PriorityQueueBasics {
    public static void main(String[] args) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        minHeap.offer(40); minHeap.offer(10); minHeap.offer(30);
        System.out.println(minHeap.peek()); // 10
        System.out.println(minHeap.poll()); // 10

        PriorityQueue<Integer> maxHeap =
            new PriorityQueue<>(Collections.reverseOrder());
        maxHeap.offer(40); maxHeap.offer(10); maxHeap.offer(30);
        System.out.println(maxHeap.peek()); // 40
    }
}
```

Custom comparator: comparator returns negative when a should come before b, positive when after, zero when equal. Prefer Integer.compare(a, b), not a - b, to avoid integer overflow.

```
import java.util.*;

class Task {
    int id, priority;
    Task(int id, int priority) {
        this.id = id;
        this.priority = priority;
    }
}

public class CustomPriorityQueue {
    public static void main(String[] args) {
        PriorityQueue<Task> pq =
            new PriorityQueue<>((a, b) ->
                Integer.compare(a.priority, b.priority));

        pq.offer(new Task(1, 30));
        pq.offer(new Task(2, 10));
        pq.offer(new Task(3, 20));

        while (!pq.isEmpty()) {
            Task t = pq.poll();
            System.out.println(t.id + " -> " + t.priority);
        }
    }
}
```

Note: PriorityQueue iteration is not sorted. Only peek/poll guarantee access to the current highest-priority element. To print in priority order, repeatedly poll a copy or the queue itself.

Heap index formulas (0-based array)

• parent(i) = (i - 1) / 2

• left(i) = 2*i + 1

• right(i) = 2*i + 2

• Min-heap: parent <= children

• Max-heap: parent >= children

• Heap height = O(log n); peek O(1); insert/delete-root O(log n).

7. Implement a binary heap from scratch

This is a min-heap backed by an array. Insert appends then sifts up; removeMin replaces root with last element then sifts down.

```
import java.util.*;

public class MinHeap {
    private final List<Integer> a = new ArrayList<>();

    public int size() { return a.size(); }
    public boolean isEmpty() { return a.isEmpty(); }
```

```
    public void offer(int x) {
        a.add(x);
        siftUp(a.size() - 1);
    }

    public int peek() {
        if (a.isEmpty()) throw new NoSuchElementException();
        return a.get(0);
    }

    public int poll() {
        if (a.isEmpty()) throw new NoSuchElementException();
        int answer = a.get(0);
        int last = a.remove(a.size() - 1);
        if (!a.isEmpty()) {
            a.set(0, last);
            siftDown(0);
        }
        return answer;
    }

    private void siftUp(int i) {
        while (i > 0) {
            int p = (i - 1) / 2;
            if (a.get(p) <= a.get(i)) break;
            Collections.swap(a, p, i);
            i = p;
        }
    }

    private void siftDown(int i) {
        int n = a.size();
        while (true) {
            int left = 2 * i + 1, right = 2 * i + 2;
            int smallest = i;
            if (left < n && a.get(left) < a.get(smallest)) smallest = left;
            if (right < n && a.get(right) < a.get(smallest)) smallest = right;
            if (smallest == i) break;
            Collections.swap(a, i, smallest);
            i = smallest;
        }
    }

    public static void main(String[] args) {
        MinHeap h = new MinHeap();
        h.offer(7); h.offer(2); h.offer(9); h.offer(1);
        while (!h.isEmpty()) System.out.print(h.poll() + " ");
        // 1 2 7 9
    }
}
```

8. Monotonic Stack

A monotonic stack maintains values in non-decreasing or non-increasing order. It is used when each element needs its next/previous greater or smaller element.

Next Greater Element to the right — O(n)

```
import java.util.*;

public class NextGreaterElement {
    static int[] nextGreater(int[] a) {
        int n = a.length;
        int[] ans = new int[n];
        Arrays.fill(ans, -1);
        Deque<Integer> st = new ArrayDeque<>(); // indices

        for (int i = n - 1; i >= 0; i--) {
            while (!st.isEmpty() && a[st.peek()] <= a[i]) st.pop();
            if (!st.isEmpty()) ans[i] = a[st.peek()];
            st.push(i);
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            nextGreater(new int[]{4, 5, 2, 10, 8})));
        // [5, 10, 10, -1, -1]
```

```
    }
}
```

Why store indices? Indices let you calculate distances/widths and access original values. For just values, storing values can be enough.

Daily Temperatures — distance to next warmer day

```
import java.util.*;

public class DailyTemperatures {
    static int[] solve(int[] t) {
        int n = t.length;
        int[] ans = new int[n];
        Deque<Integer> st = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            while (!st.isEmpty() && t[i] > t[st.peek()]) {
                int j = st.pop();
                ans[j] = i - j;
            }
            st.push(i);
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            solve(new int[]{73,74,75,71,69,72,76,73})));
        // [1, 1, 4, 2, 1, 1, 0, 0]
    }
}
```

Largest Rectangle in Histogram — O(n)

```
import java.util.*;

public class LargestRectangleHistogram {
    static int largestRectangle(int[] h) {
        Deque<Integer> st = new ArrayDeque<>();
        int best = 0, n = h.length;

        for (int i = 0; i <= n; i++) {
            int current = (i == n) ? 0 : h[i];
            while (!st.isEmpty() && h[st.peek()] > current) {
                int height = h[st.pop()];
                int leftSmaller = st.isEmpty() ? -1 : st.peek();
                int width = i - leftSmaller - 1;
                best = Math.max(best, height * width);
            }
            st.push(i);
        }
        return best;
    }

    public static void main(String[] args) {
        System.out.println(largestRectangle(new int[]{2,1,5,6,2,3})); // 10
    }
}
```

Implementation note: The sentinel iteration at i == n flushes remaining bars. The code stores n as a sentinel index but never reads h[n].

9. Monotonic Deque — Sliding Window Maximum

For each window of size k, keep indices in decreasing order of their values. Remove expired indices from the front; remove smaller/equal values from the back.

```
import java.util.*;

public class SlidingWindowMaximum {
    static int[] maxSlidingWindow(int[] a, int k) {
        if (a == null || k <= 0 || k > a.length) return new int[0];

        int n = a.length;
        int[] ans = new int[n - k + 1];
        Deque<Integer> dq = new ArrayDeque<>();
```

```
        for (int i = 0; i < n; i++) {
            while (!dq.isEmpty() && dq.peekFirst() <= i - k)
                dq.pollFirst();

            while (!dq.isEmpty() && a[dq.peekLast()] <= a[i])
                dq.pollLast();

            dq.offerLast(i);

            if (i >= k - 1)
                ans[i - k + 1] = a[dq.peekFirst()];
        }
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            maxSlidingWindow(new int[]{1,3,-1,-3,5,3,6,7}, 3)));
        // [3, 3, 5, 5, 6, 7]
    }
}
```

Complexity: O(n) time and O(k) auxiliary space. Every index enters and leaves the deque at most once.

10. Top-K and Kth-element patterns with PriorityQueue

Kth largest element — keep a min-heap of size k

```
import java.util.*;

public class KthLargest {
    static int findKthLargest(int[] nums, int k) {
        if (k < 1 || k > nums.length) throw new IllegalArgumentException();
        PriorityQueue<Integer> pq = new PriorityQueue<>();

        for (int x : nums) {
            pq.offer(x);
            if (pq.size() > k) pq.poll();
        }
        return pq.peek();
    }

    public static void main(String[] args) {
        System.out.println(findKthLargest(new int[]{3,2,1,5,6,4}, 2)); // 5
    }
}
```

Why min-heap? Keep only the k largest seen values. The smallest among those k sits at the root; that is the kth largest overall.

Top K frequent elements

```
import java.util.*;

public class TopKFrequent {
    static int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : nums) freq.put(x, freq.getOrDefault(x, 0) + 1);

        PriorityQueue<Integer> pq = new PriorityQueue<>(
            (a, b) -> Integer.compare(freq.get(a), freq.get(b)));

        for (int x : freq.keySet()) {
            pq.offer(x);
            if (pq.size() > k) pq.poll();
        }

        int[] ans = new int[Math.min(k, freq.size())];
        for (int i = ans.length - 1; i >= 0; i--) ans[i] = pq.poll();
        return ans;
    }

    public static void main(String[] args) {
        System.out.println(Arrays.toString(
            topKFrequent(new int[]{1,1,1,2,2,3}, 2)));
    }
}
```

Complexity: O(n + m log k), where m is the number of distinct values; space O(m + k).

Merge K sorted arrays/lists — core heap idea

• Put the first item from each sorted list into a min-heap as (value, listId, index).

• Repeatedly poll the smallest; append it to result; insert the next item from that same list.

• If total number of elements is N and there are k lists: O(N log k) time, O(k) heap space.

Find K closest points / K closest numbers

• Use a max-heap of size k keyed by distance, so the farthest among the kept k is removed first.

• For points: squared distance = x*x + y*y; square root is unnecessary for comparison.

• Time O(n log k), space O(k). Use long for squared distance if coordinate limits can overflow int.

11. BFS, level order and shortest path connection

Queue is the natural structure for BFS because it processes nodes in first-discovered-first-processed order.

Binary tree level-order traversal

```
import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public class LevelOrderTraversal {
    static List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> result = new ArrayList<>();
        if (root == null) return result;

        Queue<TreeNode> q = new ArrayDeque<>();
        q.offer(root);

        while (!q.isEmpty()) {
            int levelSize = q.size();
            List<Integer> level = new ArrayList<>();

            for (int i = 0; i < levelSize; i++) {
                TreeNode node = q.poll();
                level.add(node.val);
                if (node.left != null) q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
            result.add(level);
        }
        return result;
    }

    public static void main(String[] args) {
        TreeNode root = new TreeNode(1);
        root.left = new TreeNode(2);
        root.right = new TreeNode(3);
        root.left.left = new TreeNode(4);
        System.out.println(levelOrder(root)); // [[1], [2, 3], [4]]
    }
}
```

Critical BFS rule: For an unweighted graph, mark visited when enqueuing, not after dequeuing, to avoid adding the same node many times. Complexity O(V + E) with adjacency lists.

12. 0-1 BFS and scheduling patterns

0-1 BFS

For a graph whose edge weights are only 0 or 1: use a deque. If traversing an edge of weight 0, add the neighbor to the front; if weight 1, add to the back. This is a specialized shortest-path method.

• Initialize dist[source] = 0 and others = infinity.

• Relax edge u→v with weight w: if dist[u] + w < dist[v], update dist[v].

• For w=0: addFirst(v); for w=1: addLast(v).

• Typical complexity O(V + E).

Task scheduling with priorities

• If next task is chosen by urgency/priority, use PriorityQueue rather than FIFO queue.

• If tasks must preserve arrival order among equal priorities, add a sequence number as a tie-breaker.

• If tasks can be cancelled/updated, Java PriorityQueue does not support efficient arbitrary key updates; use lazy deletion or a TreeMap / indexed heap depending on requirements.

13. Complexity cheat sheet

Operation	ArrayDeque	LinkedList	PriorityQueue
Insert at front/back	O(1) amortized	O(1)	—
Remove at front/back	O(1)	O(1)	—
Peek front/top	O(1)	O(1)	peek head O(1)
Offer / insert priority	—	—	O(log n)
Poll / remove priority head	—	—	O(log n)
Search contains(x)	O(n)	O(n)	O(n)
Remove arbitrary value	O(n)	O(n)	O(n)

Complexities are typical for Java's standard implementations. ArrayDeque grows its backing array when needed; PriorityQueue is a binary heap. LinkedList has node-allocation and pointer overhead.

14. Common interview mistakes

• Using Stack when ArrayDeque is the preferred modern Java stack abstraction.

• Calling pop(), remove(), or element() on an empty structure without checking.

• Assuming PriorityQueue iteration returns sorted order — it does not.

• Using (a, b) -> a - b as a comparator; subtraction can overflow. Use Integer.compare(a, b).

• Confusing a max-heap with Java's default min-heap.

• Storing values instead of indices in monotonic stack/deque problems when distance or window expiry is needed.

• Not removing expired indices in sliding-window deque.

• Using <= versus < incorrectly in monotonic stack; duplicates change whether equal values are popped.

• Not handling empty input, k <= 0, k > n, or null where allowed.

• Marking BFS visited only after dequeue, which can cause duplicate enqueues.

15. Interview problem bank — solve in this order

Core stack:

• Valid Parentheses

• Min Stack

• Implement Stack using Queues

• Implement Queue using Stacks

• Evaluate Reverse Polish Notation

• Basic Calculator / Basic Calculator II

• Decode String

• Simplify Path

• Asteroid Collision

• Remove All Adjacent Duplicates in String

• Online Stock Span

Monotonic stack:

• Next Greater Element I and II

• Daily Temperatures

• Next Smaller Element

• Previous Greater/Smaller Element

• Largest Rectangle in Histogram

• Maximal Rectangle

• Sum of Subarray Minimums

• Sum of Subarray Ranges

• 132 Pattern

• Trapping Rain Water (two pointers and stack approaches)

Queue and deque:

• Implement Queue using Array

• Implement Circular Queue

• Design Circular Deque

• Sliding Window Maximum

• First Negative Integer in Every Window

• Moving Average from Data Stream

• Number of Recent Calls

• Rotting Oranges

• Binary Tree Level Order Traversal

• Shortest Path in Binary Matrix

• 0-1 BFS problems

Heap / PriorityQueue:

• Kth Largest Element in an Array

• Top K Frequent Elements

• K Closest Points to Origin

• Find K Pairs with Smallest Sums

• Merge K Sorted Lists

• Find Median from Data Stream

• Task Scheduler

• Reorganize String

• Meeting Rooms II

• Smallest Range Covering Elements from K Lists

• IPO / maximize capital

• Last Stone Weight

• Sort Characters by Frequency

• Kth Smallest Element in a Sorted Matrix

Design / advanced:

• LRU Cache (HashMap + Deque/doubly linked list)

• LFU Cache (HashMap + frequency lists)

• Max Stack

• Design Browser History

• Design Twitter feed (heap merge)

• Sliding Window Median (two heaps + lazy deletion)

• Time-based priority scheduling

• Indexed priority queue / decrease-key design

Suggested order: Stack basics → Queue/Deque → BFS → monotonic stack → monotonic deque → heap basics → top-K → merge-K → two heaps → design problems.

16. Pattern recognition: problem dekho, structure choose karo

Problem clue	Likely structure	Reason
Most recent / undo / nested	Stack	LIFO order
First come, first served / BFS	Queue	FIFO order
Both ends matter / window	Deque	Efficient front and back
Repeatedly get min/max priority	PriorityQueue / heap	Root is best candidate
Next greater/smaller / span	Monotonic stack	Discard dominated candidates
Max/min for every fixed-size window	Monotonic deque	Maintain best candidate in window
K largest/smallest in stream	Heap of size K	Keep only K candidates
Merge many sorted streams	Min-heap	Always extract global smallest

17. Final formula sheet

• Stack = LIFO; Queue = FIFO; Deque = both ends; PriorityQueue = priority order.

• Heap parent/children (0-indexed): parent=(i-1)/2, left=2i+1, right=2i+2.

• Heap insert / remove root = O(log n); peek = O(1); heapify = O(n).

• Monotonic stack: each index pushed once and popped once ⇒ O(n).

• Sliding window maximum: deque stores decreasing values' indices; expire if index <= i-k; answer at i >= k-1.

• Histogram: area = height × (rightBoundary - leftBoundary - 1).

• Kth largest: min-heap size k; Kth smallest: max-heap size k.

• Top K: heap size k ⇒ commonly O(n log k) after frequency counting.

• BFS adjacency list: O(V+E); unweighted shortest path via BFS.

• 0-1 BFS: weight 0 → addFirst; weight 1 → addLast.

Readiness checkpoint: You should be able to explain why each structure is chosen, write its operations without looking, state its invariant, dry-run edge cases, and justify time/space complexity. Then solve the problem bank progressively.